

Java RMI (Remote Method Invocation)

Master 1 CSIA

Introduction

- L'interface de programmation RMI (Remote Method Invocation) se focalise sur le principe orienté objet et permet l'appel distant des objets en utilisant leur méthodes. Elle se base donc sur une communication de plus haut niveau qui consiste à définir une couche offrant des services plus complexes (appelée middleware ou inter-logiciel).
- Cette couche est réalisée en s'appuyant sur la couche TCP/UDP qui devient cette fois-ci transparente au développeur en le comparant avec l'interface Socket.

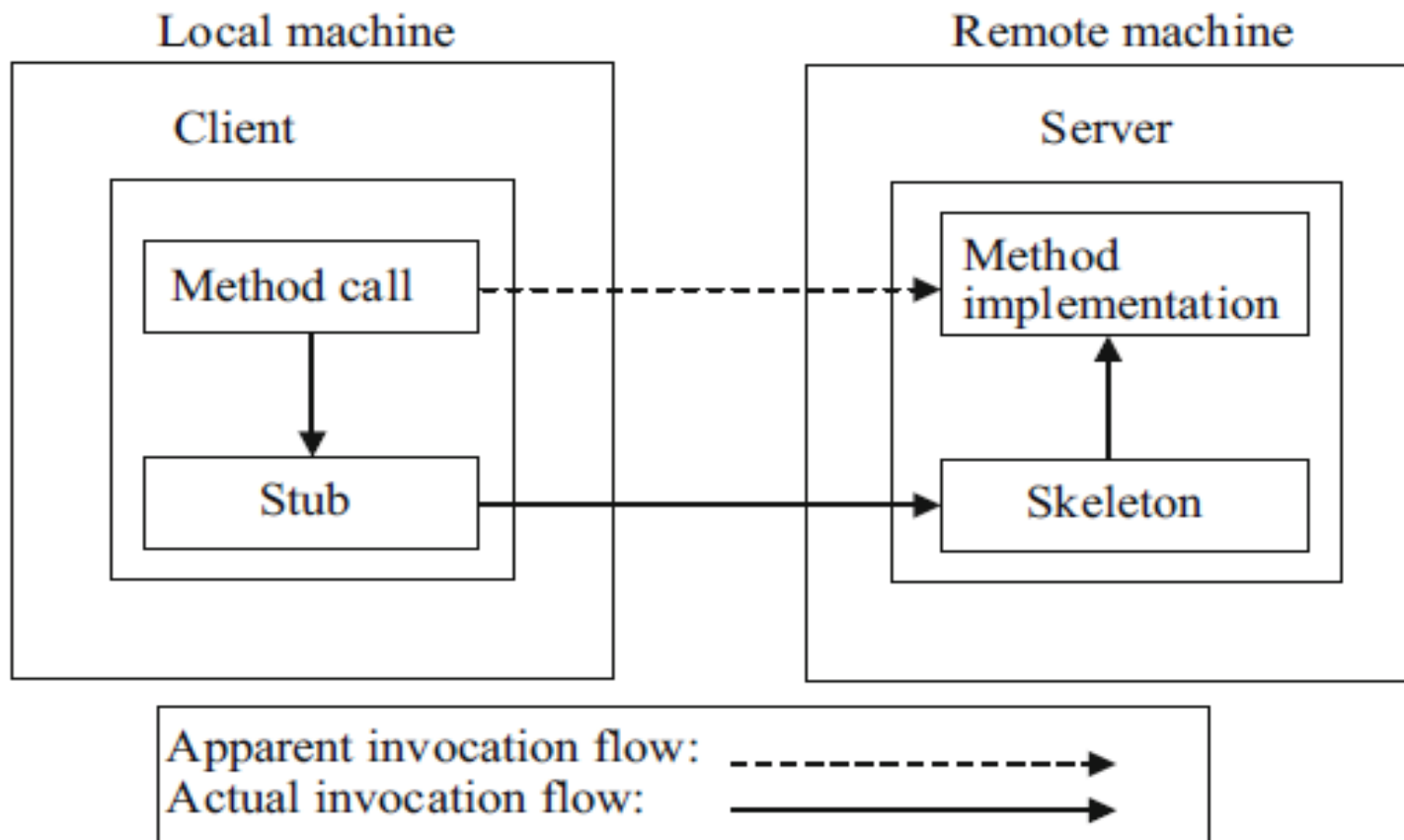
Java RMI

- La communication via RMI se fait via l'appel de méthodes entre les objets Java qui s'exécutent sur des machines virtuelles différentes (espaces d'adressage distincts), sur le même ordinateur ou sur des ordinateurs distants reliés par un réseau.
- RMI utilise directement les sockets et les échanges se font avec un protocole propriétaire: RMP (Remote Method Protocol)

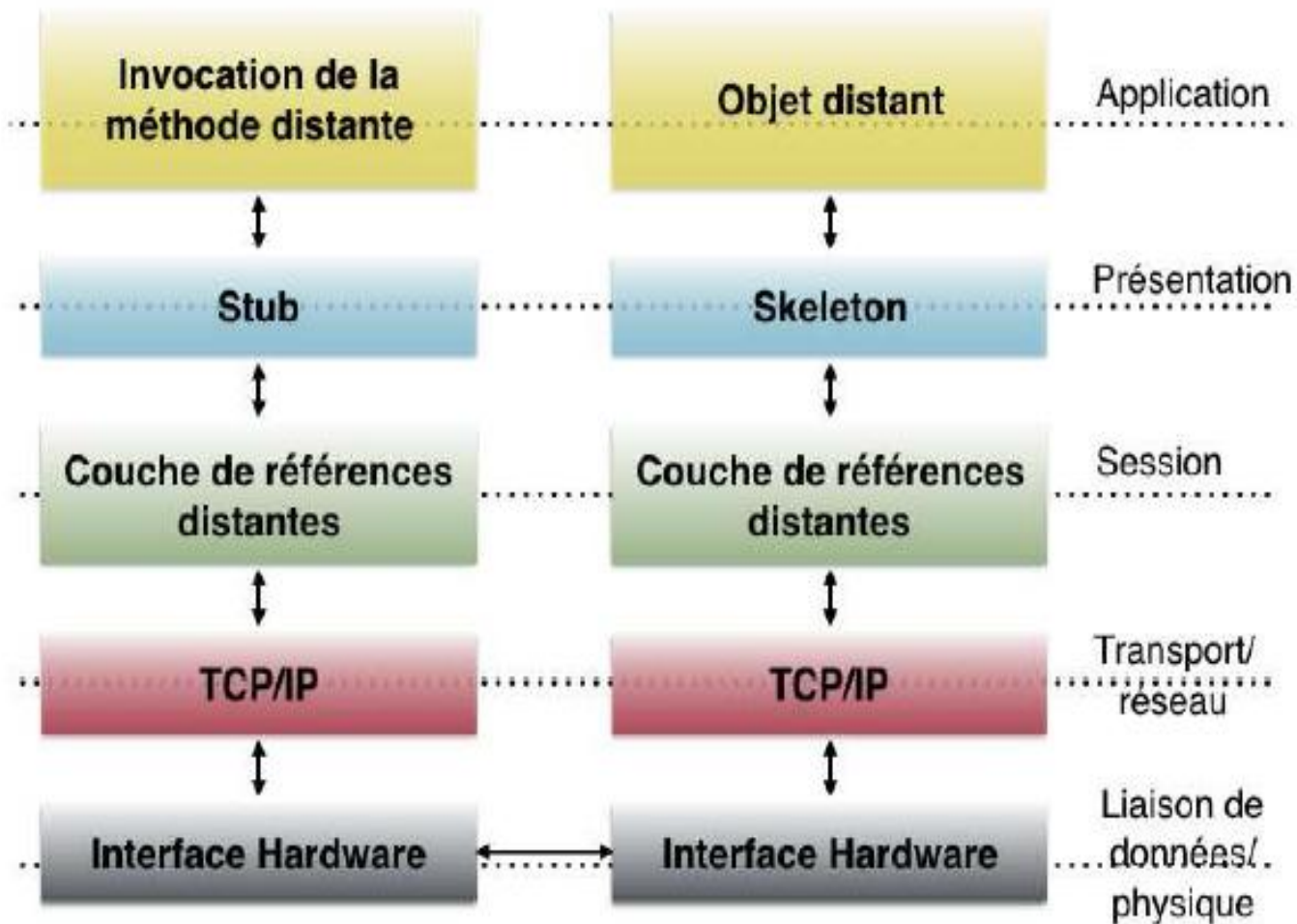
Objectifs

- Les objectifs d'utilisation de cette interface de programmations sont:
 - Rendre transparente la manipulation d'objets distants par le principe de délégation connue par le patron de conception « proxy ». Ce patron consiste à considérer une couche intermédiaire pour communiquer deux entités logicielles.
 - Utiliser les interfaces Java pour masquer au programmeur l'appel d'objets distants et faciliter l'utilisation de ces objets.
 - Préserver la sécurité via RMI Security Manager.

L'architecture d'une application client/serveur utilisant java RMI



L'architecture d'une application client/serveur utilisant java RMI



Architecture et principe de fonctionnement de RMI

- Le serveur contient les objets distants et le client de son côté fait appel à ses objets en invoquant ses méthodes. Les principales couches de cette architecture sont :

Les amorces (Stub/Skeleton)

- afin de communiquer un client et un serveur en RMI, il faut définir deux amorces (proxy) Stub et Skeleton. Ces amorces assurent le rôle d'adaptateurs pour le transport des appels distants sur la couche réseau. Elles réalisent également l'assemblage et le désassemblage des paramètres (sérialisation, désérialisation). Les amorces Stub et Skeleton sont créés par le générateur **rmic** appelé via une ligne de commande et chacun possède un rôle particulier

Stub (côté Client)

- C'est le représentant local de l'objet distribué. Il initie une connexion avec la JVM distante en transmettant l'invocation à « la couche des références d'objets ». Il assemble les paramètres pour leur transfert à la JVM distante puis se met en attente des résultats de l'invocation distante. Une fois la réponse est reçu, il désassemble des valeurs et renvoie la valeur à l'appelant.

Skeleton (côté Serveur)

- permet de désassembler les paramètres pour la méthode distante et fait appel à la méthode demandée. Une fois les résultats sont obtenus, il effectue l'assemblage du résultat à destination de l'appelant.

La couche des références d'objets

- permet d'obtenir une référence d'objet distribué à partir de la référence locale au Stub. Cette fonction est assurée grâce à un service de noms **rmiregister** qui joue le rôle de service d'annuaire pour les objets distants enregistrés. Un unique **rmiregister** est défini par JVM qui accepte des demandes de service sur le port **1099**.

La couche transport

- Elle connecte les deux espaces d'adressage (JVM) et gère les connexions en cours (écoute et répond aux invocations). Elle emploie un protocole **JRMP (Java Remote Method Invocation)** basé sur TCP/IP.
- JRMP a été modifié en supprimant la nécessité des Skeletons. Depuis la version 1.2 de Java, une même classe Skeleton générique est partagée par tous les objets distants.

Développement d'une application RMI

Afin de développer une application RMI, il faut suivre les étapes suivantes :

1. Définir une interface Java pour un objet distant.
2. Créer une classe implémentant cette interface
3. Créer une application serveur RMI qui gère les objets distants.
4. Créer une application côté client RMI.
5. Compiler les fichiers précédents
6. Créer les classes Stub et Skeleton (commande **rmic**).
7. Démarrer **rmiregistry** et lancer l'application serveur RMI.
8. Lancer le programme client accédant à des objets distants du serveur

Exemple d'une application client/serveur en RMI

- Dans cet exemple, le serveur propose des services de calcul sous forme de calculatrice distante. Les opérations à implémenter sur le serveur sont: l'addition, la soustraction, la multiplication et la division.
- Pour implémenter cette architecture client/serveur, il suffit de créer les classes et interfaces suivantes:

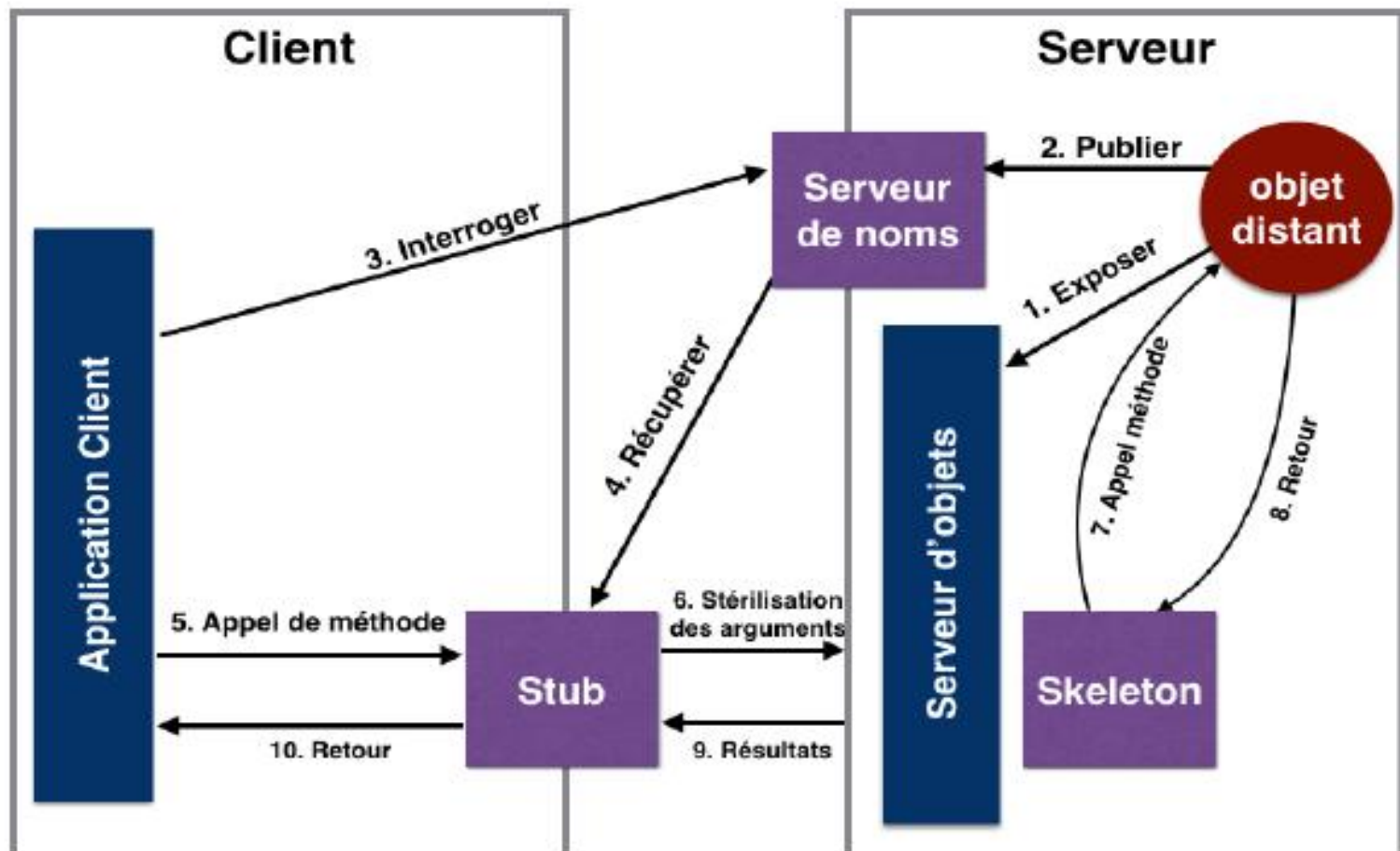
Exemple d'une application client/serveur en RMI

- L'interface Calculator.java
- L'objet serveur de calcul CalculatorImpl.java
- Le programme serveur CalculatorServer.java
- Le programme client CalculatorClient.java

détails de chaque étape

- Définir une interface Java et son implémentation côté serveur pour un objet distant.
- Tout objet qui désire être exposé à travers les RMI doit implémenter l'interface **java.rmi.Remote**
- Cette interface doit contenir des méthodes dont la signature contient une clause throws faisant apparaître l'exception **java.rmi.RemoteException**.
- Les paramètres ou les valeurs de retour doivent être sérialisables, i.e. implémentent l'interface **java.io.Serializable**.

Processus de communication entre un client et un serveur RMI



1. Calculator.java

```
import java.rmi.*;

public interface Calculator extends Remote {
    public long add(long a, long b) throws RemoteException;
    public long sub(long a, long b) throws RemoteException;
    public long mul(long a, long b) throws RemoteException;
    public long div(long a, long b) throws RemoteException;
}
```

2. CalculatorImpl.java

L'implémentation de l'interface qui est une spécialisation de la classe
java.rmi.server.UnicastRemoteObject

```
import java.rmi.*;
import java.rmi.server.*;
public class CalculatorImpl extends UnicastRemoteObject implements Calculator {

    // il faut surcharger le constructeur vide pour déclarer l'exception RemoteException
    public CalculatorImpl() throws RemoteException {super(); }
    public long add(long a, long b) throws RemoteException{return a + b;}
    public long sub(long a, long b) throws RemoteException{return a - b;}
    public long mul(long a, long b) throws RemoteException{return a * b;}
    public long div(long a, long b) throws RemoteException{return a / b;}

}
```

3. Le serveur RMI: CalculatorServer.java

- L'enregistrement d'un objet peut s'effectuer à l'aide de la méthode statique: **void bind(String nom, Remote o)** de la classe **java.rmi.Naming** tel que :
- **nom** est de la forme **machine:port/id** (machine et port sont optionnels).
- **o** est l'objet à exposer.
- **id** c'est un nom/identifiant du service offert par l'objet à exposer.

3. CalculatorServer.java

- En pratique, il faut utiliser la méthode **void rebind(String nom, Remote o)** au lieu de bind pour que l'objet soit réexposer en cas de duplication.

3. CalculatorServer.java

```
import java.rmi.Naming;
public class CalculatorServer {
    public CalculatorServer() {
        try {
            Calculator c = new CalculatorImpl();

            Naming.rebind("rmi://localhost:1099/CalculatorService", c);
        }
        catch (Exception e) {System.out.println("erreur: " + e); }
    }
    public static void main(String args[]) { new CalculatorServer(); }
}
```

4. CalculatorClient.java

- Pour atteindre l'objet, côté client, il suffit d'interroger le registre via la méthode statique **(Object) lookup(String nom)** de la classe **Naming**. Puis utiliser l'objet ordinairement pour y appeler des méthodes.

4. CalculatorClient.java

```
import java.rmi.Naming;
public class CalculatorClient{
    public static void main(String[] args) {
        try {
            Calculator c = (Calculator) Naming.lookup( "rmi://localhost/CalculatorService");
            System.out.println("addition (4,5): " + c.add(4, 5));
            System.out.println("multiplication (3,6):" + c.mul(3, 6));
        } catch (MalformedURLException murle) {
            System.out.println("java.net.MalformedURLException" + murle);
        } catch (RemoteException re) {
            System.out.println("java.rmi.RemoteException");
        } catch (NotBoundException nbe) {
            System.out.println("java.rmi.NotBoundException" + nbe);
        } catch (java.lang.ArithmeticException ae) {
            System.out.println("java.lang.ArithmeticException" + ae);
        }
    }
}
```


5. Compiler les fichiers et générer les amorces

- **>javac Calculator.java**
- **>javac CalculatorImpl.java**
- **>javac CalculatorServer.java**
- **>javac CalculatorClient.java**

- Puis, générer les amorces (Stub et Skeleton) à l'aide du service rmic comme suit:
- **>rmic CalculatorImpl**

5. Compiler les fichiers et générer les amorces

- Ceci génère le fichier CalculatorImpl-Stub.java et CalculatorImpl-Skel.java (n'est pas généré dans v1.2 ou plus).
- Il compile les fichiers et supprime les fichiers sources (.java). Pour garder les sources, il suffit d'ajouter l'option keep:
- **>rmic keep**

6. Enregistrer l'objet dans l'annuaire

- Pour que l'objet soit atteignable par un client, il faut l'enregistrer dans un annuaire des objets exposés. Le service réseau correspondant à cet annuaire est fourni par défaut avec l'environnement Java, pour le démarrer :
- **>rmiregistry** ou bien, **>rmiregistry port** qui permet de lancer l'annuaire sur un autre port que celui qui est attribué par défaut (1099).

7. Lancer le serveur puis le client

- **>java CalculatorServer**
- **>java CalculatorClient**

Conclusion

- Ce cours a présenté une interface de programmation réseau en Java qui repose sur l'appel distant des méthodes.
- Elle utilise le principe d'orienté objet pour structurer les services et les appels distants.
- Même si cette interface semble plus facile que les Socket Java en raison de transparence de l'utilisation de la couche transport, elle se limite par la communication des applications orientées objet uniquement ce qui n'est pas supporté par tous les langages de programmation existants.

TD/TP N°6

- **Exercice** : Ecrire un programme en mode client/serveur qui utilise l'interface de programmation RMI pour communiquer un objet distant « Service ». Cet objet possède deux méthodes distantes:
puissance (int a,int b) qui retourne a puissance b.
nbChiffres (int c) qui retourne le nombre de chiffres de l'entier c. Par exemple si c=34234 le nombre de chiffres est 5.